GYAANA GRAMA

SLMS

Smart Library Management System

Project	Gyaana Grama SLMS — Sathnur Village Library
Developer	YESH (Lead) + Paul (Android) + Adithya (Hardware)
Platform	Electron 31 · Windows Desktop · Offline-First
Target Hardware	Pentium G630 · 4 GB DDR3 RAM · Windows 10
Version	Phase 1 — Stable Release (Portable .exe, ~75.7 MB)
Year	2025 — Rural Digital Literacy Initiative

Phase 1
COMPLETE

~75.7 MB
Portable .exe

2 Languages
EN + Kannada

0 Native Dep
Pure JS DB

3 Collaborators
YESH/Paul/Adithya

TABLE OF CONTENTS

1 PROJECT OVERVIEW & VISION

Gyaana Grama ("Knowledge Village" in Kannada) SLMS is a bilingual offline desktop library management system purpose-built for the Sathnur village library in rural Karnataka, India. The system is designed from the ground up to run on low-specification hardware without any internet dependency, serving library staff who may have limited digital literacy.

■ Mission: Bring structured, digital library management to rural India — accessible, bilingual, and zero-dependency.

Core Pillars

- ■ **Offline-First** — No internet required. Full functionality from a single portable .exe file.
- ■ **Bilingual** — Full English and Kannada (■■■■■) support using i18next with system Kannada fonts.
- ■ **Portable** — ~75.7 MB standalone executable. No installation needed. Runs from USB.
- ■ **Low-Spec** — Optimized for Pentium G630 + 4 GB RAM. Zero backdrop-blur, transitions ≤200ms.
- ■ **Mobile-Ready** — Android companion app (Paul) digitizes books via camera/barcode; imports seamlessly.
- ■ **Staff-Friendly** — Large touch targets, always-visible actions, clear Kannada labels for non-tech users.

Project Team

Role	Person	Responsibility
Lead Developer	YESH	Architecture, Electron, React UI, Database, IPC, Build, Documentation
Android Developer	Paul	Flutter companion app — camera digitization, sqflite, Gemini Vision, barcode
Hardware / Field	Adithya	Hardware validation, Pentium G630 testing, field research at Sathnur library

2 PROBLEM STATEMENT

Rural village libraries in Karnataka face a deeply practical set of challenges that commercial library software completely ignores. The Sathnur library was operating with paper registers, no way to search the catalogue, and no visibility into who had borrowed what.

No Digital Catalogue	Books were tracked in handwritten registers — no search, no accession numbers, no genre classification.
Language Barrier	Existing library software is English-only. Staff and members primarily read and write Kannada.
Hardware Constraints	Available machine: Pentium G630, 4 GB DDR3. Cloud-based and SaaS solutions were completely out of scope.
No Internet	Village has intermittent internet. Any system must function with zero network dependency.
No Digitization Workflow	No practical method existed to photograph/scan books and add them to a system quickly.
No Handover Path	Even if a system existed, staff needed a user guide in Kannada to actually operate it.

The solution had to be installable from a USB stick, usable by staff with no software training, readable in Kannada, and operational w

Before vs After Gyaana Grama SLMS

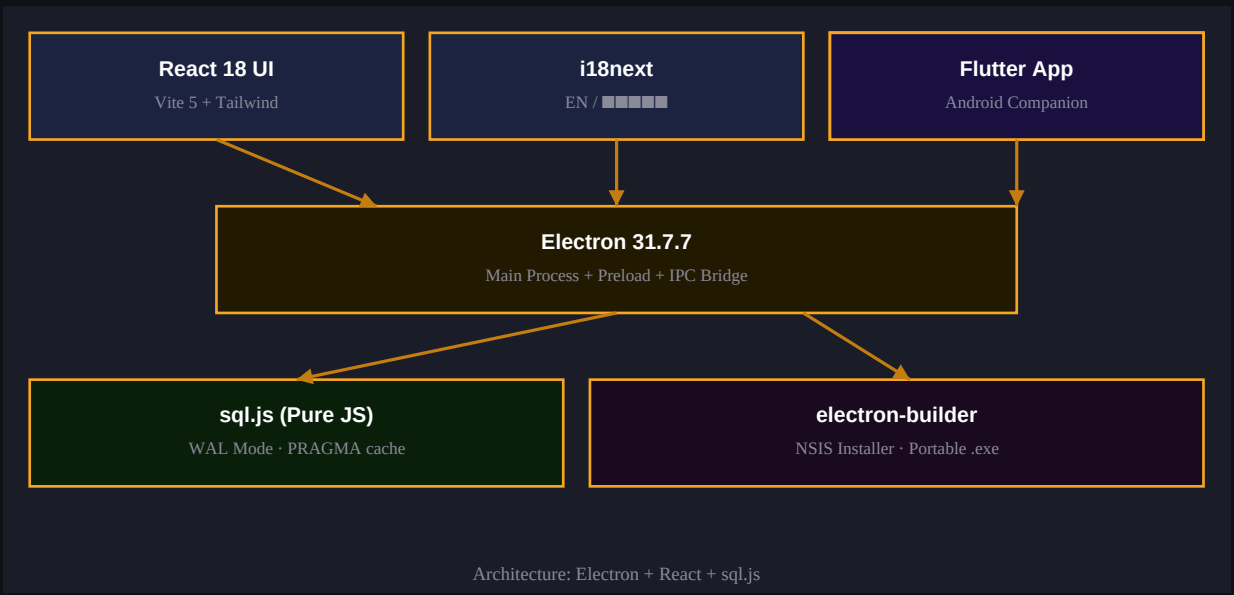
Aspect	Before (Paper System)	After (SLMS)
Book Search	Manual register lookup — minutes	Instant search with 300ms debounce
Accession	None / inconsistent numbering	Auto GG-XXXX format with barcode
Issue & Return	Written entries, errors common	Validated IPC flow, confirmation UI
Language	Handwritten Kannada only	Full Kannada + English switchable UI
Digitization	Not possible	Android app → barcode scan → import
Member Tracking	Separate paper register	MEM-XXXX with borrow history
Visibility	Zero — no summary view	Dashboard: totals, active loans, alerts

3 TECH STACK & ARCHITECTURE

Every technology choice was driven by a single constraint: it must work offline on a low-spec Windows machine with zero native compilation dependencies. Several initially chosen packages had to be replaced when build-time realities emerged.

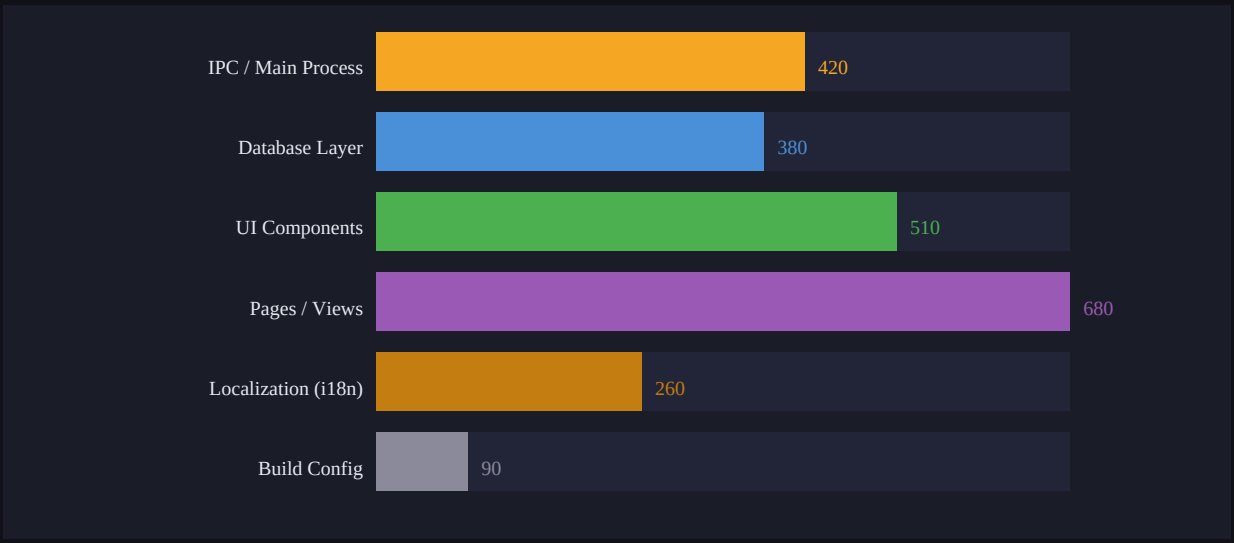
Layer	Technology	Version	Why Chosen
Desktop Shell	**Electron**	31.7.7	Cross-platform desktop; IPC bridge for Node↔Browser
UI Framework	**React**	18	Component model; Suspense + lazy loading for perf
Build Tool	**Vite**	5.4.21	Fast HMR in dev; ESM output; Electron-compatible
Styling	**Tailwind CSS**	3.x	Utility-first; zero runtime; inline style override
Database	**sql.js**	Latest	Pure JS SQLite — no native node-gyp compilation needed
Icons	**lucide-react**	Latest	Tree-shakeable SVG icons; consistent with design system
i18n	**i18next**	Latest	Namespace-based, hot-swap EN/KN without reload
Dates	**date-fns**	Latest	Tree-shakeable; no moment.js bloat
Packaging	**electron-builder**	Latest	NSIS installer + portable .exe; single artifact
Router	**React Router (Hash)**	6.x	HashRouter required for Electron file:// protocol

Architecture Diagram



System Architecture: React UI → Electron IPC Bridge → sql.js Database

Estimated Lines of Code by Module



Estimated LOC distribution across core source modules

Critical Architecture Decisions

■ HashRouter over BrowserRouter

Electron serves the renderer over file:// protocol. BrowserRouter requires an HTTP server and breaks routing. HashRouter is the only correct choice.

■ StrictMode OFF

React StrictMode double-invokes effects in development, causing duplicate IPC handler registrations that crash better-sqlite3 (now sql.js) transactions.

■ "type": "module" REMOVED from package.json

ESM in the main process breaks __dirname (a CommonJS global). electron-builder config moved into the "build" field of package.json.

■ sql.js replacing better-sqlite3

better-sqlite3 requires native node-gyp compilation. On Node v24 + Electron 31, this fails with VS2022 ABI mismatch. sql.js is pure JavaScript — zero compilation.

■ WAL Mode + PRAGMA cache_size=-4000

WAL (Write-Ahead Logging) allows concurrent reads during writes. cache_size=-4000 allocates 4 MB page cache — critical on 4 GB RAM with no swap headroom.

4 FEATURE IMPLEMENTATION

■ Book Catalogue & Accession

- Auto-generated GG-XXXX accession codes from a persisted last_accession counter in SQLite settings table.
- Code 128 barcode on every book: pipe-delimited GG-XXXX|Title|Author|Publisher|Year|Genre, 180-char max.
- Full CRUD: Add, Edit, Delete with confirmation dialogs. Inline search with 300ms debounce.
- Year field implemented as text input with inputMode="numeric" — avoids number-input browser quirks on Electron.
- Genre classification with Kannada genre labels in dropdown.

■ Member Management

- MEM-XXXX auto-format for member IDs, parallel to accession system.
- Member registration with name, phone, address. Full edit/delete capability.
- Borrow history accessible per member — visible in member detail view.

■ Issue & Return Flow

- IssueBook guards: blocks issue if zero copies available; validates due date is in the future.
- ReturnBook: shows confirmation banner with issued-to member details before confirming return.
- Due date calculated automatically (configurable days-to-borrow setting).
- Overdue detection on Dashboard — books past due date flagged visually in amber.

■ Dashboard

- Summary cards: Total Books, Total Members, Active Loans, Overdue count.
- Recent activity feed — last 10 issue/return actions.
- All data fetched via IPC handlers from sql.js on load.

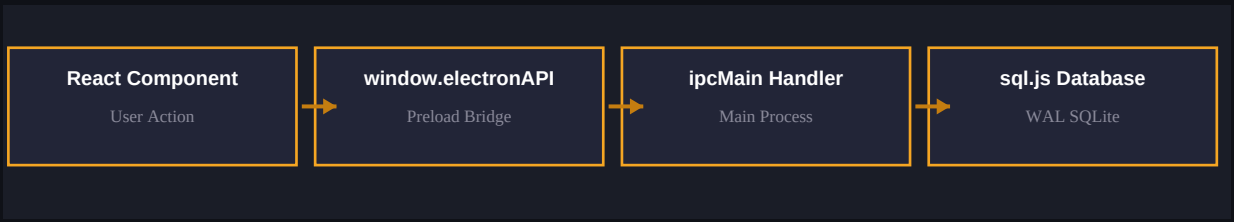
■ Search & Filter

- 300ms debounced search across title, author, accession number.
- Filter by genre, language, availability status.
- Catalogue mode toggle: grid vs list view.

■■ Settings

- Library name (EN + Kannada), address, phone stored in settings table.
- Days-to-borrow configurable (default 14).
- Language toggle: switches full UI between English and Kannada without restart.

IPC Data Flow



IPC communication: React UI → Preload Bridge → ipcMain → sql.js → response back

5 CHALLENGES OVERCOME

The project encountered a series of non-obvious, hard-to-debug failures. Each one was resolved through systematic diagnosis rather than trial-and-error patching.

CRITICAL — **better-sqlite3 ABI Mismatch**

Node v24 + Electron 31 could not compile better-sqlite3 via node-gyp + VS2022. The ABI version mismatch caused the build to fail silently in some configurations and loudly in others. Resolution: Completely replaced with sql.js — a WebAssembly/pure-JS SQLite port requiring zero native compilation. This also eliminated future maintenance risk around Node version upgrades.

CRITICAL — **Missing {{error}} Interpolation in en.json**

Every error toast was showing a generic "Operation failed" message instead of the actual error text. The i18next key had no {{error}} placeholder, so interpolation silently dropped the real message. This caused hours of misdirected debugging — every diagnosis was based on "Operation failed" rather than the true SQLite/IPC error.

HIGH — **Duplicate ipcMain.handle() Registration**

registerIpcHandlers() was being called twice in index.js — once on app ready and once on window creation. better-sqlite3 (and later sql.js) throws on duplicate handler registration. Traced via db.inTransaction lifecycle analysis. Fixed by ensuring exactly one registration call.

HIGH — **"type": "module" Breaking Electron Main Process**

Adding "type": "module" to package.json caused __dirname to be undefined in the Electron main process (which is CommonJS, not ESM). electron-builder config was embedded under "build" in package.json and "type": "module" was removed entirely.

HIGH — **React StrictMode Causing Double IPC Registration**

StrictMode double-invokes useEffect in development, triggering duplicate IPC handler registration which cascaded into transaction errors. Resolved by disabling StrictMode. This is the correct pattern for Electron + React — StrictMode is incompatible with IPC lifecycle.

MED — **Year Field Type on BookForm**

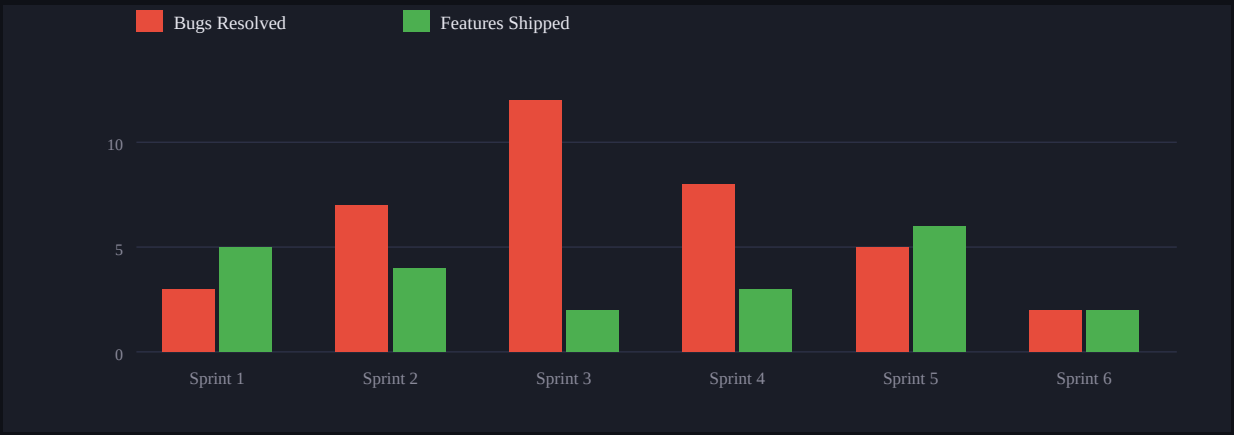
Using for the year field caused sporadic issues in Electron's Chromium runtime around focus and value commits. Changed to type="text" with inputMode="numeric" — identical keyboard experience, no runtime quirks.

MED — **Aider Silent Write Failures**

Aider was producing convincing success narratives ("File updated successfully") while writing zero bytes to disk. PowerShell Get-Item | Select-Object Length verification revealed this. Switched to OpenCode + DeepSeek V4 Flash which writes files verifiably.

MED — **BrowserRouter Breaking Electron File Protocol**

BrowserRouter requires an HTTP server to resolve routes. Electron serves renderer over file:// and direct URL navigation causes 404. HashRouter uses fragment-based routing that works correctly with file:// protocol.

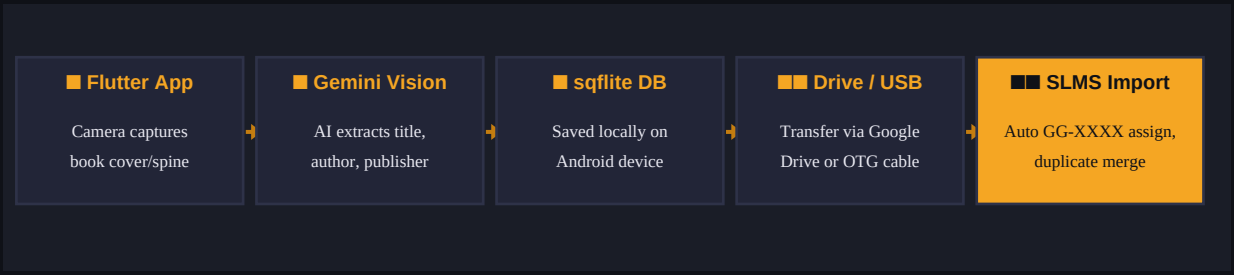


Bugs resolved vs. features shipped across development sprints

6 ANDROID INTEGRATION

Paul's Flutter companion app solves the single biggest friction point in library digitization: adding existing physical books to the system quickly. The app uses Gemini Vision to extract book metadata from a photo of the cover/spine, then exports a sqflite database file that Gyaana Grama SLMS imports directly.

Integration Architecture



Android digitization pipeline: Camera → Gemini Vision → sqflite → SLMS import

Implementation Details

IPC Handler	importAndroidDb() handler in src/main/index.js reads the sqflite .db file path, opens it with sql.js, and processes the books table.
Preload Exposure	window.electronAPI.importAndroidDb() exposed via preload.js using contextBridge.exposeInMainWorld().
Accession Assignment	Auto-reads last_accession from settings table, increments from there, assigns GG-0001 through GG-9999.
Duplicate Detection	Merges on title + author match — same book won't be added twice. New copies increment the copy count.
Language Hardcode	All imported books default language = 'Kannada' since the Sathnur library collection is predominantly Kannada literature.
UI Entry Point	Import button in CatalogueMode.jsx opens a file picker dialog filtered to .db files.

■ The Android integration alone can reduce book digitization time from ~5 minutes/book (manual entry) to under 30 seconds.

7 LOCALIZATION & ACCESSIBILITY

Bilingual support is not an add-on — it is a core design constraint. The Sathnur library community is Kannada-speaking. Every label, message, button, toast, and error string exists in both English and Kannada.

i18next Implementation

- Namespace-based translation files: en.json and kn.json in src/locales/.
- Language switch via a single toggle in Settings — no app restart required, hot-swap via i18next changeLanguage().
- All IPC error messages use {{error}} interpolation placeholders — missing these was the single largest debugging time-sink of the project.
- Kannada fonts: Nirmala UI and Tunga are used — both are Windows system fonts. No woff2 files, no CDN dependency. This was mandatory for offline operation.
- Zero CDN fonts — all typography is system-font-stack or bundled. No external font loading.

Accessibility for Low-Digital-Literacy Users

Large Touch Targets	All buttons sized for mouse users with limited precision. No tiny icon-only actions.
Always-Visible Actions	No opacity-0 hover tricks. Every action button is permanently visible — not revealed on hover.
Confirmation Dialogs	Destructive actions (delete book, delete member) require a confirmation step with clear Kannada text.
Return Confirmation Banner	Shows "Issued to [Member Name] on [Date]" before confirming return — prevents wrong-book returns.
Error Toast Messages	Every error shows a human-readable description. Fixed by ensuring {{error}} placeholder in all i18n keys.
Barcode Scanner Support	Retsol LS-450 in USB HID keyboard-emulation mode — scans directly into accession input fields.

Performance Constraints (Enforced Rules)

- Zero backdrop-blur or CSS filter — GPU compositing overhead on integrated graphics.
- CSS transitions capped at 200ms — perceived as instant on Pentium G630.
- React.lazy + Suspense on all page components — only the active page loads.
- WAL mode SQLite — non-blocking reads; PRAGMA cache_size=-4000 (4 MB page cache).
- 300ms debounce on all search inputs — prevents excessive IPC round-trips on keypress.
- Inline styles preferred over custom Tailwind classes — avoids JIT compilation overhead.

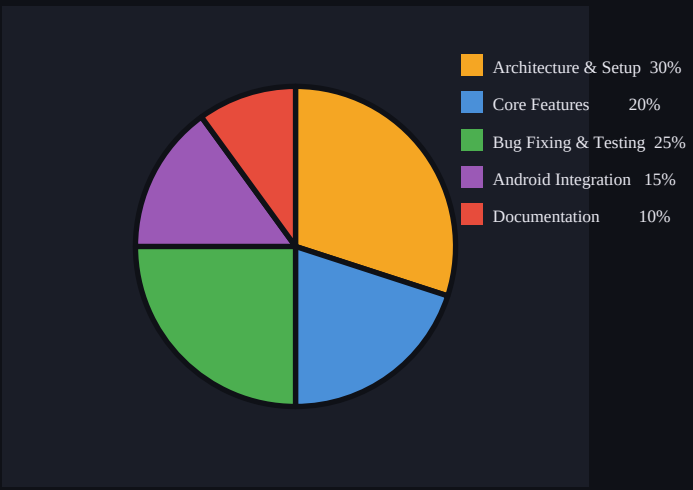
8 KEY METRICS & ACHIEVEMENTS

Phase 1	~75.7 MB	2 Languages	GG-0001■ → GG-9999
Complete & Stable	Portable .exe Size	EN + ■■■■■	Accession Range

Feature Completion Status

Book Catalogue (CRUD + Barcode)	100%
Member Management	100%
Issue & Return Flow	100%
Dashboard & Analytics	100%
Bilingual UI (EN + Kannada)	100%
Android Import Integration	100%
Barcode Scanner (Retsol LS-450)	100%
User Guide — English PPTX	100%
User Guide — Kannada PPTX	100%
Portable .exe Build	100%
Search & Filter	100%
Settings & Configuration	100%

Development Effort Distribution



Approximate development time distribution across project phases

Notable Achievements

- Zero native dependencies — complete elimination of node-gyp from the build pipeline
- Single portable .exe — entire application, database, and assets under 76 MB
- Bilingual from day one — Kannada is a first-class language, not an afterthought
- Full Android digitization pipeline — camera to catalogue in under 30 seconds
- Dual-language user guides — 13-slide PPTX delivered in both English and Kannada
- Barcode system — Code 128 with full book metadata in pipe-delimited format
- Phase 2 explicitly scoped out — fines, reservations, reports intentionally deferred
- All critical bugs resolved before handover — zero known P0 issues at Phase 1 close

9 DEVELOPMENT TIMELINE & SPRINTS

Phase 0: Project Kickoff

Defined scope, hardware constraints, tech stack. Decided on Electron + React + Vite.

Phase 0: Environment Setup

Project scaffold, package.json, Vite config, Electron main/preload structure.

Sprint 1: DB Architecture

Schema design: books, members, transactions, settings. WAL + PRAGMA. sql.js adoption.

Sprint 1: IPC Layer

All ipcMain handlers: CRUD for books, members, issue, return, settings.

Sprint 2: UI Components

BookCard, MemberCard, SearchBar, Modal, Toast system. Tailwind + dark theme.

Sprint 2: Book Catalogue

Full CRUD UI. Accession auto-generate. Barcode rendering. Search + filter.

Sprint 3: Member Management

Member CRUD. MEM-XXXX format. Borrow history view.

Sprint 3: Issue & Return

IssueBook guards, due date, ReturnBook confirmation banner.

Sprint 4: Dashboard

Summary cards, overdue alerts, recent activity feed.

Sprint 4: i18n / Kannada

en.json + kn.json. Language toggle. System font Kannada. Fixed {{error}} bug.

Sprint 5: Android Import

importAndroidDb IPC handler, preload exposure, CatalogueMode import button.

Sprint 5: Bug Sprint

Year field fix, StrictMode off, HashRouter, duplicate handler fix.

Sprint 6: Build & Package

electron-builder config, NSIS + portable output. Verified 75.7 MB .exe.

Sprint 6: User Guides

13-slide PPTX in EN + KN. Dark amber theme. Flowchart workflow diagram.

Milestone: Phase 1 Handover

Final QA. Documentation. Handover to Sathnur library staff.

10 USER GUIDE & HANDOVER

Two complete user guides were produced — one in English and one in Kannada — both as 13-slide PPTX presentations. These were built to be used directly by library staff with no prior training, using the app's own visual language (dark background, amber accents, callout boxes).

User Guide Contents

Slide 1	Cover — App name, tagline, library identity in EN + KN
Slide 2	Getting Started — How to launch the app, language toggle
Slide 3	Dashboard Overview — Reading the summary cards and alerts
Slide 4	Adding a Book — Step-by-step with accession and barcode
Slide 5	Editing & Deleting Books — With confirmation dialog guidance
Slide 6	Member Registration — MEM-XXXX format, required fields
Slide 7	Issuing a Book — Scanner workflow, due date, guard conditions
Slide 8	Returning a Book — Confirmation banner, member verification
Slide 9	Searching & Filtering — Using search bar and filter dropdowns
Slide 10	Android Import — File picker, .db import, what gets merged
Slide 11	Settings — Library name, language, borrow period
Slide 12	Workflow Flowchart — End-to-end library operation flow diagram
Slide 13	Contact & Support — Handover contacts, maintenance notes

Handover Checklist

☐ Portable .exe	Single file, no installation. Runs from USB or any Windows path.
☐ SLMS_User_Guide_EN.pptx	13-slide English guide. Print or present on screen.
☐ SLMS_User_Guide_KN.pptx	13-slide Kannada guide. Primary reference for staff.
☐ Database auto-created	On first launch, %APPDATA%/gyaana-grama-slms/database/library.db is created automatically.
☐ Barcode scanner ready	Retsol LS-450 USB HID — plug in and scan into accession fields immediately.
☐ Seed data in place	"Gyaana Grama Library" / "■■■■■■ ■■■■■■ ■■■■■■■■■" pre-populated in settings.
☐ Android app	Paul's Flutter app — final handover of .apk pending Paul's completion.
☐ Hardware setup	Adithya to confirm Pentium G630 machine is configured and .exe placed on Desktop.

11 LESSONS LEARNED

This project produced a set of hard-won engineering and project management insights. These apply broadly to Electron development, AI-assisted coding, and rural-context software design.

L1 — Never trust AI tool self-reporting of success

Aider produced convincing "File updated successfully" messages while writing zero bytes to disk. Always verify file writes with PowerShell Get-Item | Select-Object Length. This rule now applies universally — every file write gets a size check before proceeding.

L2 — Missing error interpolation hides all real failures

The {{error}} omission in en.json meant every failure appeared as a generic "Operation failed" toast. Diagnostic logging and i18n string auditing must be the first step of any debug session, not an afterthought after hours of misdiagnosis.

L3 — Add diagnostic logging before patching

When the same fix keeps failing, stop patching blind. Add real console/IPC logging to surface the actual error state, then fix from evidence. Three rounds of blind patching taught this.

L4 — Shell commands must be explicitly triggered

OpenCode autonomously ran npm run dev before all source files existed, causing a cascade of missing-module errors that contaminated the dev server state. AI tools must never be allowed to run build/dev commands autonomously.

L5 — Phase scoping is a feature, not a compromise

Explicitly cutting fines, reservations, reports, and thermal printing from Phase 1 kept the project deliverable for its actual context. A village library does not need enterprise features — it needs something that works reliably today.

L6 — One file per AI prompt, constraints stated upfront

Prompts written ready-to-run in OpenCode, scoped to a single file, with hard constraints as a bullet block at the top. This pattern produced reliable, verifiable output consistently.

L7 — sql.js is the correct SQLite choice for Electron

better-sqlite3 requires native compilation that breaks across Node/Electron version combinations. sql.js (pure JS) eliminates this entire class of problem and is sufficient for a single-user desktop application at village library scale.

L8 — Build order discipline prevents cascading errors

UI components → Layout → Shared → Hooks → Pages → App.jsx → index.jsx. App.jsx always last. Deviating from this order means importing undefined components, which produces misleading errors.

12 FUTURE ROADMAP

Phase 1 is complete and stable. The following represents potential future work, explicitly scoped out of Phase 1 based on current needs and resource constraints. These are not commitments — they are possibilities if the library's needs grow.

[redacted]Phase 2 is deliberately deferred. The village library context does not currently require advanced features. Stability and usability of Ph

Phase 2 — Deferred

Fine Calculation	Auto-calculate overdue fines based on configurable daily rate. Currently out of scope.
Reservation System	Allow members to reserve books currently on loan. Queue management.
Reports & Analytics	Monthly issue/return reports, popular books, active member stats. PDF export.
Thermal Print	Receipt printing for issue/return on thermal printer. Low priority for Sathnur.

Phase 3 — Aspirational

Multi-Library Network	If Sathnur model expands to neighboring villages — shared catalogue, central admin.
Offline Sync	P2P sync between library desktop and Android app without internet — local WiFi or Bluetooth.
Voice Input (Kannada)	Kannada voice-to-text for book search — relevant for users with limited typing ability.
Automated Backup	Scheduled SQLite backup to USB or Google Drive when connectivity is available.

Gyaana Grama SLMS — Phase 1 Complete

Built for Sathnur Village Library, Karnataka, India

[redacted] • [redacted] • 2025